# Creator Fee Safety — in plain English

If you launch a token on Polydash, you earn a **creator fee** (0% – 2%, you choose; UIs default to 1%) on every
trade of your token — first on the bonding curve, and after graduation on the Uniswap V4 pool. This document explains
who can touch that money (only you) and why claiming it is safe.

## The short version

- Your fee rate is **locked when you launch**. Nobody — not you, not Polydash — can change it later.
- Your fees can **only ever be sent to your payout wallet**. There is no other destination in the code.
- Polydash (the factory owner/multisig) has **no function** that can pause, redirect, seize or reduce your fees.
- Changing your payout wallet takes **48 hours**, and your current payout wallet can **veto** the change.
- Claiming is **pull-based**: the contract sends you ETH. It never asks for approvals, signatures or tokens.

## How fees accrue

| Phase | Where your fees sit | How you get them |
|---|---|---|
| Bonding curve | `Polydash-Launch.creatorFeesAccrued` (ETH, inside your launch contract) | `Polydash-Launch.claimCreatorFees()` |
| After graduation (Uniswap V4) | `Polydash-Hook.pools[poolId].creatorFeesAccrued` (ETH, inside the hook) | the same `claimCreatorFees()` call also pulls these |

Fees are always ETH — never your own token — so claiming never sells your token or moves its price.

## Claiming

`claimCreatorFees()` can be called by **anyone** (you, a bot, a friend). It does not matter who calls it: the ETH
always goes to `creatorPayoutWallet`. This means you can automate claims without giving anyone control.

```
amount = creatorFeesAccrued;
creatorFeesAccrued = 0;                 // state updated BEFORE sending (checks-
effects-interactions)
_sendEth(creatorPayoutWallet, amount);  // the only possible destination
```

## Changing your payout wallet (48h timelock)

1. **Creator** calls `initiatePayoutWalletChange(newWallet)` → starts a 48-hour countdown (event emitted).
2. During those 48h, **the creator or the current payout wallet** can call `cancelPayoutWalletChange()`.
   So if your creator key is stolen, the attacker can't quietly redirect your fees — your payout wallet sees the
   event and vetoes it.
3. After 48h, **anyone** can call `executePayoutWalletChange()`. Everything earned up to that moment is paid to the
   **old** wallet first; only fees earned afterwards go to the new one. If the old wallet can't receive ETH, those fees
   roll over instead of getting stuck, and the change still goes through.

`address(0)` is rejected as a payout wallet, so fees can never be burned by mistake.

## Re-entrancy protection

Every function that moves ETH (`claimCreatorFees`, the payout-wallet functions, `buy`, `sell`, `commit`, `uncommit`,
`claimRefund`, `collectPlatformFees`) is protected by OpenZeppelin's `ReentrancyGuard`, and balances are zeroed before
ETH is sent. Tests (`test/unit/CreatorFee.t.sol`, `test/unit/Reentrancy.t.sol`) use malicious payout wallets that try
to re-enter during a claim — the re-entry reverts and no extra ETH leaves the contract.

## No admin override

Search the code for any function that writes `creatorFeesAccrued` or `creatorPayoutWallet`: the only writers are
trades (which add to it), the claim (which pays it to the payout wallet), and the creator-controlled timelock above.

- The factory owner **cannot** call `claimCreatorFees` to a different address — the function has no address argument.
- `PolydashLaunch` is a non-upgradeable clone; `launchImplementation` is `immutable` in the factory.
- The hook used by your launch is fixed at creation; `setHook()` only affects **future** launches.
- Tested in `test_OwnerAndFactoryCannotRedirectOrSeizeCreatorFees` and the invariant `invariant_CreatorFeesOnlyToPayoutWallet`.

## Why a claim can’t drain your wallet

This is the question most creators ask, so here it is directly:

- **A claim only sends ETH to you.** It is a payment from the launch/hook contract to your payout wallet.
  Nothing is ever pulled from your wallet.
- **No approvals are involved.** The claim never calls `approve`, `transferFrom`, `permit` or anything that would
  let a contract spend your tokens or ETH. ( `test_ClaimNeverTakesTokensOrApprovals` checks your token balance is
  unchanged and your ETH balance only goes up.)
- **No signature is needed.** You never sign a message to claim, so there is nothing a phishing site could replay.
- **Anyone calling it for you is harmless** — the destination is hard-wired to your payout wallet.
- **The amount is bounded** by what the contract has recorded as owed to you; it can’t send more than it holds.

The real risks are the usual ones outside these contracts: a fake website asking you to sign an approval, or a
compromised private key. A genuine Polydash claim will **never** ask your wallet for an approval or a token transfer
— if it does, it is not Polydash.